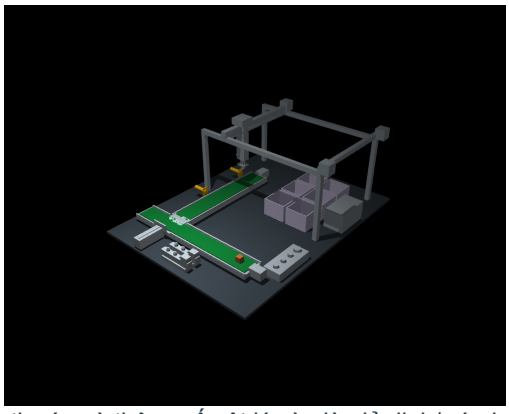
01 Bạn đang xây dựng cái gì?

Mục tiêu: kể lại được hành trình của một phôi và phân biệt máy ảo, cảm biến, điều khiển và bản sao số.

Lõi Digital Twin từ số 0 - tài liệu học 10 trang, gắn với mã và kết quả của dự án ngày 27/10/2026. Đọc theo thứ tự; không cần biết trước Python hay điều khiển tự động.

Baseline là phiên bản cơ sở để kiểm tra từng giả định trước khi thêm chức năng. Trong các bài này, Python/MuJoCo làm phần tính và điều khiển; Unity không tham gia các lượt nghiệm thu. Chương trình Python này chưa nối với PLC thật.

Hãy tưởng tượng một kiện hàng chạy trên băng. A1 đẩy nó sang băng thứ hai; PS1/PS2 báo khi nó đi qua. Robot XY đưa đầu hút tới vị trí gắp; A2 hạ/nâng đầu hút; robot mang hàng tới một trong sáu hộp.



Ảnh mô hình mô phỏng hiện có. Kích thước và thông số vật lý còn là giả định/ước lượng.

Khái niệm	Hiểu bằng ngôn ngữ thường
Plant / đối tượng	Chiếc máy cần điều khiển; ở giai đoạn này là máy ảo MuJoCo.
Cơ cấu chấp hành	Bộ phận nhận lệnh và tạo chuyển động: động cơ, xi lanh, hút.
Cảm biến	Nguồn thông tin máy có thể đọc, chẳng hạn có/không có phôi.
Bộ điều khiển	Chọn lệnh tiếp theo dựa trên điều kiện và thông tin đang có.
Bản sao số	Biểu diễn số có cập nhật thông tin để hỗ trợ theo dõi/ước lượng/dự báo. Lõi hiện chưa đủ mọi chức năng này.

Vì sao hình 3D chưa đủ?

Hình 3D mô tả hình dạng. Mô phỏng vật lý còn cần khối lượng, lực, ma sát và tiếp xúc. MuJoCo tính chuyển động từng bước; trong mô hình này, băng dùng vận tốc bề mặt và giác hút dùng lực bám tiếp xúc giả định. Chưa có mô hình áp suất/rò khí đã hiệu chuẩn.

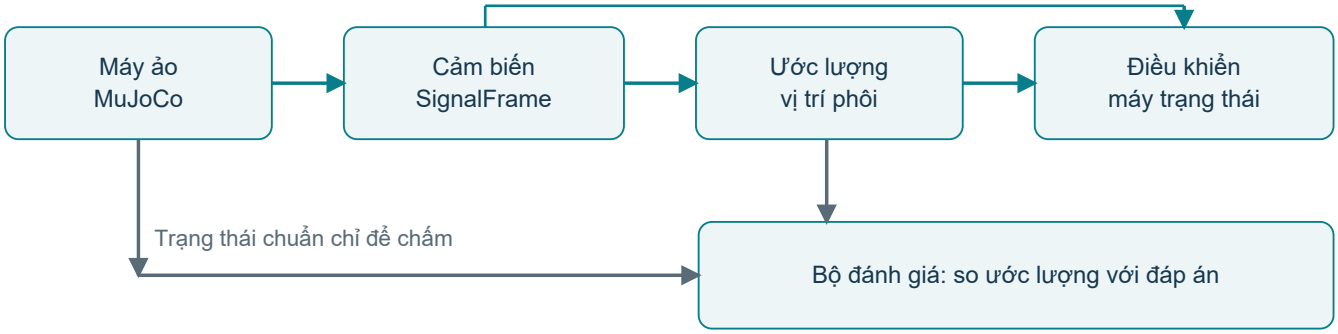
Tự kiểm tra: Đầu hút chuyển động đẹp có chứng minh đã gắp được phôi không?
Đáp án: Không. Phải có bằng chứng phôi đi theo đầu hút hoặc kiểm tra độc lập; lệnh bật hút chỉ là một lệnh.

Đối chiếu: mujoco/README.md; production_cell.xml; run_cell.py

Số đo trong tài liệu là mô phỏng, trừ khi được ghi rõ khác.

02 Dữ liệu đi qua chương trình ra sao?

Mục tiêu: chỉ ra ai được đọc tín hiệu, ai được đọc đáp án và một vòng lặp Python làm gì.



Controller nhận cả tín hiệu trực tiếp lẫn ước lượng. MuJoCo biết vị trí thật của mọi vật vì nó đang tính mô phỏng; máy thật không tự cung cấp “tọa độ chính xác của phôi”. Nếu controller đọc thẳng đáp án đó, bài thử dễ thành công nhưng chưa chứng minh cách điều khiển có thể chuyển sang máy thật.

Tệp trong mujoco/	Bạn đọc nó để hiểu
run_cell.py	Máy ảo và commissioning cũ; commissioning dùng đáp án phôi.
signals.py + sensor_model.py	Cấu trúc gói dữ liệu và cách tạo tín hiệu cảm biến.
estimator.py	Từ các mốc quang suy ra vị trí và vận tốc.
sequence.py	Điều kiện chuyển bước, timeout và HOLD.
run_twin.py	Ghép các phần, phát lệnh trở lại máy ảo và ghi log.

Đọc Python bằng một ví dụ nhỏ

```
frame, edges = sensor.sample(state)
command = controller.tick(frame, frame.received_time_s)
# Sau đó bộ nối áp dụng command vào máy ảo.
# Chỉ evaluator dùng state để tính sai số.
```

Dấu **=** gán kết quả cho biến; dấu chấm gọi một chức năng của đối tượng. **sample** tạo một mẫu; **tick** xử lý một lượt điều khiển. Dictionary là bảng tên-giá trị, ví dụ **{"ps1": True}**. **if** nghĩa là “nếu điều kiện đúng thì làm việc này”.

Vòng lặp hiện đọc cảm biến, ước lượng/chọn lệnh, áp dụng lệnh, ghi log rồi tiến mô phỏng. Luồng lệnh từ controller quay lại plant. Các số sai số trong log không được đưa ngược vào quyết định canh gắp.

Tự kiểm tra: sensor_model được đọc vị trí khớp để tạo công tắc; controller có được đọc cùng dữ liệu đó không?
Đáp án: Không. Mô hình cảm biến nằm bên trong plant; đầu ra công khai cho controller là tín hiệu nhị phân.

03 Cảm biến thực sự nói được điều gì?

Mục tiêu: đọc đúng True, False, None và hiểu vì sao hạ 60 mm không phải hết hành trình 75 mm.

Cảm biến quang trong mô hình được tạo từ tia đo khoảng cách, sau đó đổi thành có/không phát hiện. **ps1=True** không phải tọa độ liên tục; nó chỉ cho biết vùng tia đang có vật gây phát hiện. Mô hình hiện là cảm biến lý tưởng, chưa mô phỏng mọi nguyên nhân báo sai.

Tín hiệu	Ý nghĩa / trạng thái hiện tại
ps1, ps2	Hai vị trí phát hiện phôi trên băng tải dài.
a1_retracted / extended	A1 gần đầu thu 0 mm / đầu ra 110 mm.
a2_retracted / extended	A2 gần đầu trên 0 mm / đầu dưới 75 mm.
home_x, home_y1, home_y2	Chưa xác nhận nguồn phản hồi: None / unavailable.
vacuum_ok	Chưa có nguồn xác nhận chân không: None / unavailable.
belt_speed_*	Chưa có encoder tốc độ được xác nhận; không dùng lệnh tốc độ như phép đo.

Mô phỏng công tắc cuối hành trình

Gọi q là hành trình. Công tắc tác động nếu khoảng cách tới đầu hành trình không quá 2 mm. Dấu |...| nghĩa là lấy độ lớn, bỏ dấu âm; dung sai 2 mm là tham số thử chưa hiệu chuẩn.

```
A1 ra hết:  |q_A1 - 110 mm| <= 2 mm
A2 xuống hết: |q_A2 - 75 mm| <= 2 mm
Nếu q_A2 = 60 mm: |60 - 75| = 15 mm > 2 mm
```

Vì thế tại 60 mm, tín hiệu dưới A2 vẫn False. Ta giữ nguyên điều này. Dời ngưỡng xuống 60 chỉ để chương trình chạy tiếp sẽ che mất giới hạn thật của phép đo. A1 trong baseline mới chờ công tắc ra hết, thay vì dùng tọa độ khớp để chuyển bước sớm.

A2 trong MuJoCo nhận lệnh vị trí. Điều đó không chứng minh van khí nén 5/2 ngoài thực tế có thể giữ xi lanh tại 60 mm; đây là một giới hạn khi chuyển từ mô phỏng sang máy thật.

Giá trị / chất lượng	Cách hiểu
False / good	Có mẫu hợp lệ, mẫu đang báo chưa tác động.
None / unavailable	Chưa có nguồn đo; không thể kết luận là chưa tác động.
True hoặc False / stale	Giữ giá trị cũ nhưng đã quá tuổi cho phép.

good không có nghĩa cảm biến chắc chắn khỏe. Cảm biến kẹt mức vẫn có thể gửi mẫu đúng cấu trúc và mới, nhưng giá trị không phản ánh vật thể. Chất lượng truyền dữ liệu khác với chẩn đoán phần cứng.

Tự kiểm tra: Bật hút True có được gán vacuum_ok=True không?
Đáp án: Không. Một bên là ý định điều khiển, bên kia phải là thông tin xác nhận độc lập.

Đối chiếu: sensor_model.py; signals.py; production_cell.xml

Số đo trong tài liệu là mô phỏng, trừ khi được ghi rõ khác.

04 Thời gian, lấy mẫu và cạnh tín hiệu

Mục tiêu: hiểu 2 ms, 20 ms, timestamp, tuổi dữ liệu và vì sao một True có thể chỉ là thông tin cũ.

Đại lượng	Giá trị trong baseline	Nghĩa của nó
Bước vật lý	2 ms = 0,002 s	Khoảng thời gian mô phỏng mỗi bước giải.
Lấy mẫu / tick	20 ms = 10 bước vật lý	50 lượt mỗi giây mô phỏng.
Nguồn và nhận	source_time_s / received_time_s	Lúc lấy mẫu / lúc nhận, hiện dùng chung đồng hồ mô phỏng.
sequence	Số thứ tự tăng dần	Ghép log và nhận biết trùng hoặc đảo thứ tự.

20 ms không phải độ trễ đồng bộ với máy thật. Không có kết nối máy thật để đo đại lượng đó. Thời gian mô phỏng là thời gian của thế giới ảo; thời gian chạy trên laptop có thể nhanh hoặc chậm hơn. Hiện thời gian nhận bằng thời gian lấy mẫu, chưa có mô hình truyền mạng.

Cạnh lên là một sự kiện, không phải một mức kéo dài

```
Mẫu:      0    1    2    3    4
PS1:   False False True True False
Cạnh:     -    -   lên   -   xuống
```

Nếu đếm mọi mẫu True, một phôi có thể bị đếm nhiều lần. Chỉ chuyển False → True tạo cạnh lên để đánh dấu mốc đi qua. Mẫu đầu thiết lập trạng thái; True ngay từ đầu không chứng minh vừa có một phôi mới vượt tia.

Gói dữ liệu giống một phong bì có ngày gửi

```
{"sequence": 259,
 "source_time_s": 5.18, "received_time_s": 5.18,
 "values": {"ps1": true}, "quality": {"ps1": "good"}}
```

Đây là bản rút gọn để đọc: gói thật có đủ các tín hiệu và trường nguồn/miền đồng hồ. Trong JSON dùng **true/null**; trong Python dùng **True/None**. Dữ liệu có thể gửi được không có nghĩa dữ liệu còn mới.

```
Tuổi dữ liệu = thời gian hiện tại - thời gian lấy mẫu
Ví dụ minh họa: 5,40 - 5,18 = 0,22 s = 220 ms
```

Ngưỡng hiện tại là 100 ms: ví dụ trên thành stale dù gói vừa đến. Khoảng lấy mẫu lớn hơn 61 ms hoặc thứ tự thời gian/số thứ tự không tăng cũng đưa controller vào HOLD. Các ngưỡng này là cấu hình thử, không phải tiêu chuẩn công nghiệp chung.

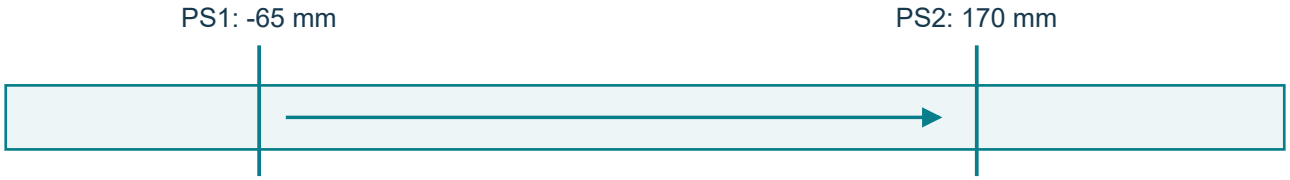
Tự kiểm tra: Gói cũ vừa nhận lúc 5,40 s có tự trở nên mới không?
Đáp án: Không. Tuổi tính từ lúc lấy mẫu. Khi sang nhiều đồng hồ thật phải xử lý thêm sai lệch đồng hồ.

Đối chiếu: signals.py; run_cell.py; sensor_model.py; sequence.py

05 Từ hai lần thấy phôi đến vị trí ước lượng

Mục tiêu: tự tính được vận tốc, vị trí tâm phôi và hiểu dấu “ước lượng” trong y_hat.

Không thấy liên tục vẫn có thể ước lượng: nếu biết vừa đi qua một mốc và chuyển động khoảng bao nhanh, ta suy ra vị trí giữa hai lần quan sát. Giống dự đoán vị trí xe từ mốc cây số và tốc độ; đó là **ước lượng**, chưa phải quan sát trực tiếp.



Khoảng cách 235 mm; phôi đi theo +Y (minh họa, không theo tỷ lệ)

Bước 1. Tính vận tốc từ hai cạnh cùng một phôi

Lượt hộp 1 ghi PS1 lên ở 5,18 s và PS2 lên ở 6,66 s. Cùng một mặt trước vượt hai tia, nên khoảng cách giữa các mốc bằng 0,235 m.

```
Delta_t = 6,66 - 5,18 = 1,48 s
v_hat = 0,235 / 1,48 = 0,15878 m/s = 158,78 mm/s
```

Hậu tố _hat trong tên v_hat/y_hat chỉ giá trị ước lượng. Vận tốc đặt là 0,16 m/s; vận tốc trung bình suy ra khoảng 0,1588 m/s. Chênh lệch có thể do lấy mẫu và động lực học mô hình, không tự chứng minh động cơ thật bị lỗi.

Bước 2. Đổi mốc mặt trước thành vị trí tâm

Phôi dài 22 mm nên tâm nằm sau mặt trước 11 mm. Lần đọc cảm biến có thể trễ trong một khoảng lấy mẫu 20 ms; mã bù xấp xỉ bằng nửa khoảng đó. Đây là phép gần đúng, không biết chính xác thời điểm vượt tia.

```
y_hat = 0,170 - 0,011 + 0,15878 * 0,020 / 2
      = 0,160588 m, khoảng 160,59 mm tại mẫu PS2
```

Bước 3. Dự đoán giữa các mốc

```
y_hat_moi = y_hat_cu + v_hat * Delta_t
Đi thêm 0,5 s: Delta_y = 0,15878 * 0,5 = 79,39 mm
```

Trước khi có PS2, mã dùng vận tốc đặt làm tiên nghiệm. Nếu PS2 đến mà chưa có PS1 hợp lệ, không tự bịa một tọa độ. Hai cạnh phải ghép đúng một phôi và lệnh tốc độ không đổi giữa chúng; thay tốc độ giữa hai mốc hiện dẫn tới từ chối mô hình vận tốc này.

Tự kiểm tra: Bỏ qua nửa chiều dài phôi gây sai theo hướng nào?
Đáp án: Nhắm quá xa về phía trước khoảng 11 mm, trước khi xét các sai số khác.

Đối chiếu: estimator.py; dt_acceptance_20261027_v2/bin_1/signals.jsonl

06 Biết mình có thể sai; đổi tọa độ thành lệnh

Mục tiêu: hiểu khoảng sai số, quãng trôi sau dừng và sự khác nhau giữa tọa độ thế giới với hành trình khớp.

Một con số 160,59 mm dễ tạo cảm giác chính xác giả. Bộ ước lượng còn giữ **uncertainty_m**: khoảng sai số kỹ thuật để quyết định có nên dùng dự báo. Đây chưa phải khoảng tin cậy 95% và chưa được hiệu chuẩn bằng máy thật.

Thao tác trong mã	Ví dụ tính tay tại PS2
Khởi tạo khoảng sai số	2 mm + 158,78 mm/s × 0,020 s / 2 = 3,59 mm.
Ngoại suy thêm 0,5 s khi băng chạy	Tăng 20 mm/s × 0,5 s = 10 mm; tổng khoảng 13,59 mm nếu chưa được hiệu chỉnh.
Dừng băng	Thêm một lần quãng trôi thử 2 mm vào vị trí và khoảng sai số.
Điều kiện canh gắp tại PS2	Khoảng sai số + phần trôi phải không vượt 10 mm.

Khi có PS2, một mốc mới hiệu chỉnh lại vị trí và khoảng sai số. Nếu chỉ giữ ngoại suy lâu, độ không chắc phải tăng. Quãng trôi 2 mm và mức tăng 20 mm/s là tham số thử; số liệu nhỏ trong một lượt không biến chúng thành một bảo đảm an toàn.

Tại sao trong mã có +0,180 và -0,120?

Trong mô hình, đầu robot ở X = -0,180 m và Y = 0,120 m khi khớp XY bằng 0. Vị trí thế giới và hành trình khớp khác nhau như vị trí một căn nhà và quãng đường đi từ nhà đó. q là hành trình khớp, đơn vị mét.

```
x_world = -0,180 + q_X    => q_X = x_dich + 0,180
y_world =  0,120 + q_Y    => q_Y = y_dich - 0,120
y_dich = 0,160588 + 0,002 = 0,162588 m
q_Y = 0,162588 - 0,120 = 0,042588 m
```

Mã hiện giả định phôi nằm trên làn X = -0,180 m, nên q_X gắp bằng 0; không ước lượng X từ PS1/PS2. Trục Z hướng lên nhưng hành trình A2 dương đi xuống theo khớp đã định nghĩa.

Vì sao thời gian XY dùng max, không cộng?

```
T_du_kien = max(|Delta_X|, |Delta_Y|) / 0,12 + 0,30
Ví dụ Delta_X=0,24 m, Delta_Y=0,12 m: T=2,30 s
```

Hai trục được ra lệnh đồng thời, nên trục lâu hơn quyết định thời gian dự kiến. Đây là thời gian từ lệnh, chưa xác nhận robot thật đã tới. Mất bước hoặc bị kẹt có thể phá vỡ giả định; vì vậy chế độ mặc định chặn trước thao tác này.

Điểm đặt là vị trí mong muốn. **Ramp** nghĩa là tăng dần điểm đặt theo tốc độ giới hạn, thay vì nhảy ngay tới đích. Khi HOLD, bộ nối giữ điểm đặt ramp đã phát; đó chưa phải phép đo vị trí đứng yên thực tế.

Tự kiểm tra: Một khoảng sai số 3,59 mm có nghĩa 95% lần gắp sẽ đúng không?
Đáp án: Không. Muốn kết luận xác suất cần mô hình sai số, dữ liệu và kiểm chứng tương ứng.

Đối chiếu: estimator.py; sequence.py; production_cell.xml

07 Máy trạng thái: làm đúng việc, đúng lúc

Mục tiêu: đọc một trình tự như danh sách công việc có điều kiện, thay vì chuỗi lệnh chờ mù.

Trạng thái trả lời "đang ở công đoạn nào?". Mỗi tick kiểm tra dữ liệu, thực hiện lệnh phù hợp rồi xét điều kiện đi tiếp. **Timeout** là hạn chờ phản hồi. Một bước chưa xong không được ngầm coi là đã xong chỉ vì bước sau cần chạy.

Trạng thái	Lệnh/chức năng	Điều kiện chuyển
INIT	Kiểm tra đầu vào	A1 thu, A2 trên, hai tia quang không che.
FEED	Băng đầu chạy	Khoảng 2,36 s, giả định một phôi tại reset.
PUSH	A1 ra 110 mm	a1_extended=True; chờ tối đa 2 s.
RETRACT	A1 thu	a1_retracted=True; chờ tối đa 2 s.
TRACK	Băng dài chạy	PS2 + đủ hai mốc + sai số cho phép.
ALIGN	XY tới vị trí ước lượng	Thời gian XY, chỉ nếu đã chọn giả định vòng hở.
LOWER	A2 hạ 60 mm	Chờ dự kiến 0,75 s, không giả tín hiệu dưới.
GRIP / LIFT	Bật hút rồi nâng	Chờ hút 0,25 s; nâng chờ công tắc A2 trên.
MOVE / LOWER_BIN	Tới hộp, hạ A2	XY theo thời gian, hạ theo thời gian.
RELEASE / RAISE	Nhả rồi nâng	Nhả 0,40 s; nâng chờ công tắc trên.
RETURN / DONE	Trở về, kết thúc	Thời gian về; evaluator chấm đặt phôi riêng.
HOLD	Dừng băng, giữ lệnh cần giữ	Không tự chạy lại khi xuất hiện tín hiệu mới.

Ví dụ chuyển PUSH sang RETRACT

```
if a1_extended:
    chuyen_sang_RETRACT()
elif thoi_gian_cho >= 2.0:
    HOLD("timeout:PUSH:a1_extended")
```

Đây là mã minh họa, không dán để chạy. Tại mỗi lượt, controller vẫn có cơ hội kiểm tra mẫu cũ hoặc lỗi phản hồi. Trong baseline hộp 1, PUSH bắt đầu 2,36 s; RETRACT bắt đầu 3,18 s; TRACK bắt đầu 4,00 s; ALIGN bắt đầu 6,66 s.

Vòng kín và vòng hở cùng tồn tại. PUSH/RETRACT dùng phản hồi công tắc; XY và A2 trung gian trong chế độ thử vẫn dựa trên thời gian. Một chu trình có vài cảm biến chưa đồng nghĩa mọi chuyển động đều được xác nhận.

Tự kiểm tra: Công tắc A1 không báo sau 2 s thì có được chạy TRACK để "thử xem" không?
Đáp án: Không. Giữ HOLD và lý do. Phải điều tra trạng thái trước khi phục hồi.

Đối chiếu: sequence.py; dt_acceptance_20261027_v2/bin_1/summary.json

08 Khi thiếu thông tin: HOLD như thế nào?

Mục tiêu: phân biệt phát hiện dấu hiệu bất thường, xác định nguyên nhân và hành động giữ.

Lỗi thử **stuck_low** giữ một tín hiệu luôn False, nhưng gói vẫn mới và quality vẫn good. Controller chỉ thấy dữ liệu đã bị thay; không được đọc nhãn lỗi hoặc đáp án trước khi quyết định. Đây là thử nghiệm có kiểm soát, không phải bằng chứng phát hiện được mọi kiểu hỏng.

Lượt thử	Phản ứng quan sát được
Chế độ mặc định	HOLD ở 6,66 s: chưa xác nhận XY/A2 trung gian/chân không. Đây là giới hạn quan sát, không phải cảm biến được chứng minh hỏng.
PS1 luôn False	HOLD ở 6,66 s: PS2 xuất hiện nhưng không có PS1, không ghép được hai mốc.
PS2 luôn False	HOLD ở 6,76 s: quá hạn chờ PS2.
A1 ra hết luôn False	HOLD ở 4,38 s: quá hạn PUSH 2 s.

Một timeout 6 giây có thể quá chậm

Nếu phôi vẫn chạy, chờ thêm có thể làm phôi rơi khỏi băng. Mã mới giới hạn thời gian theo quãng đường cho phép từ vùng chuyển phôi. Hệ số 0,44 m là tham số của cấu hình hiện tại, không phải hằng số phổ quát.

```
T_TRACK = min(6 s, 0,44 m / v_dat)
Với v_dat=0,16 m/s: T_TRACK=2,75 s
TRACK từ 4,00 s: hạn 6,75 s, tick ghi HOLD ở 6,76 s
```

HOLD không chỉ là đổi một chữ trong log

Bộ nối dừng băng, giữ các điểm đặt ramp đã phát và giữ lệnh hút đang có; không tự tắt hút nếu có thể đang mang phôi. HOLD được chốt: dữ liệu về sau không tự cho chạy tiếp. Muốn phục hồi phải xác định lại điều kiện và reset đúng cách.

Evaluator chạy thêm ít nhất 0,30 s sau HOLD để kiểm tra phôi còn được đỡ trên băng. Cả bốn trường hợp HOLD trong bộ v2 đạt kiểm tra đó. Đây chưa phải kiểm chứng mọi va chạm, mọi tình huống giữ tải hoặc chức năng an toàn của máy thật.

Phát hiện dấu hiệu khác với chẩn đoán nguyên nhân

Thiếu PS2 có thể do cảm biến hỏng, phôi bị kẹt hoặc vận chuyển chậm. Timeout cho biết giả định vận hành không được xác nhận, chưa đủ kết luận nguyên nhân. Tự tái cấu hình còn cần đánh giá phương án thay thế; baseline này mới dừng có lý do.

Tự kiểm tra: 12 lượt đạt có chứng minh phát hiện lỗi chính xác 100% không?
Đáp án: Không. Chỉ các trường hợp đã dựng đạt tiêu chí. Chưa đo tỷ lệ bỏ sót/báo sai trên nhiều lịch lỗi.

Đối chiếu: sequence.py; run_twin.py; check_twin.py; results.json (bộ v2)

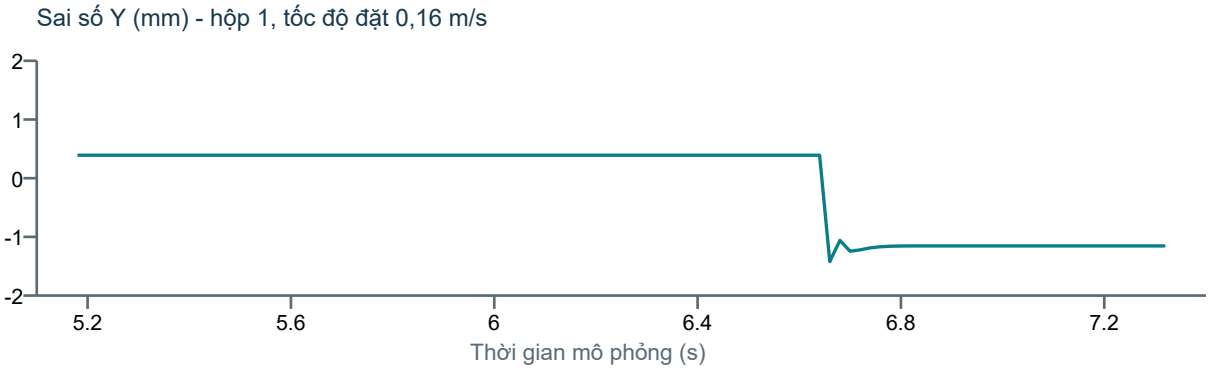
Số đo trong tài liệu là mô phỏng, trừ khi được ghi rõ khác.

09 Đọc kết quả mà không hiểu nhầm con số

Mục tiêu: đọc log, hiểu RMSE/p95 và phân biệt thành công trong mô phỏng với chất lượng hệ thật.

Mỗi lượt ghi **signals.jsonl** (cảm biến), **controller.jsonl** (quyết định/ước lượng), **ground_truth.jsonl** (đáp án chỉ để chấm) và **summary.json** (tổng kết). JSONL là mỗi dòng một gói JSON. Ghép ba log theo sequence, tránh so nhầm hai thời điểm.

Băng dài (m/s)	Chu trình hộp 1 (s)	RMSE Y (mm)	Max |sai số| (mm)
0.10	17.02	0.817	1.516
0.16	15.46	0.730	1.421
0.24	14.60	0.913	1.370



Đồ thị là sai số, không phải đường đi của phôi. Quanh 6,66 s, chương trình hiệu chỉnh tại PS2 rồi bù quãng trôi khi dừng, nên sai số có thể đổi đột ngột.

Tính sai số bằng một ví dụ nhỏ

```
e = vi_tri_uoc_luong - vi_tri_chuan
RMSE = can_bac_hai(trung_binh(e * e))
Vi du e=[1, -1, 2] mm: RMSE=sqrt(6/3)=1,414 mm
```

RMSE nhỏ khi sai số thường nhỏ; bình phương khiến sai số lớn được tính nặng hơn. **Max** là độ lớn sai số lớn nhất đã ghi. **p95** là ngưỡng mà ít nhất khoảng 95% mẫu sai số tuyệt đối trong tập đo không vượt qua; không phải xác suất đúng của mọi lần chạy tương lai.

Lượt hộp 1 có 108 mẫu sai số trong đoạn sau PS1, khi TRACK/ALIGN; RMSE 0.730 mm, p95 1.159 mm. Không đo toàn bộ XYZ hay giai đoạn chưa thấy phôi. Đây là so với đáp án của mô phỏng, chưa so với máy thật.

14 kiểm tra logic kiểm tra quy tắc; **12 lượt nghiệm thu** chạy vật lý và chấm đầu ra. Tám lượt đặt phôi và bốn lượt HOLD đều đạt. Các lượt xác định này không đủ để tuyên bố độ tin cậy thống kê. DONE và đặt phôi đúng cũng phải được đọc riêng.

Tự kiểm tra: Chu trình mới dài hơn commissioning cũ có phải thuật toán thất bại không?
Đáp án: Chưa thể kết luận: tiêu chí chờ và giả định đã khác. Giai đoạn này chứng minh baseline, chưa chứng minh tăng năng suất.

Đối chiếu: run_twin.py; dt_acceptance_20261027_v2/results.json và bin_1/ground_truth.jsonl

10 Tự chạy, tự giải thích, rồi đi tiếp

Mục tiêu: chạy ba thí nghiệm, đọc đúng kết quả và biết chính xác phần nào còn chưa làm.

1. Mở PowerShell từ Start trên Windows. WSL là môi trường Linux chạy trong Windows; MuJoCo đã cài ở đó trên máy này. Dán từng dòng sau; các biến chỉ là tên ngắn cho đường dẫn, không phải lệnh điều khiển máy thật.

```
cd C:\Users\Admin\Documents\ChatGPT\plc
$py = "/home/admin/dt-cpps-venv/bin/python"
$root = "/mnt/c/Users/Admin/Documents/ChatGPT/plc"
$app = "$root/mujoco/run_twin.py"
```

2. Chạy chế độ mặc định. Mong đợi HOLD với lý do thiếu xác nhận XY/A2/chân không, sau khi đã theo dõi phôi. Đây là kết quả có chủ đích.

```
$out = "$root/output/hoc_strict_01"
wsl -d Ubuntu-24.04 -- $py $app --output $out
```

3. Chạy trọn mô phỏng có giả định vòng hở. Cửa sổ tự chuyển động; không cần bấm Run. Đóng cửa sổ sau khi xong để nhận tổng kết. Mong đợi DONE và placement_pass_evaluator_only=true.

```
$out = "$root/output/hoc_nominal_01"
wsl -d Ubuntu-24.04 -- $py $app --output $out --assume-open-loop --view
```

4. Giả lập PS2 kẹt thấp. Mong đợi HOLD với timeout:TRACK:ps2; xem hold_time_s và kiểm tra phôi còn được đỡ sau dừng.

```
$out = "$root/output/hoc_loi_01"
wsl -d Ubuntu-24.04 -- $py $app --output $out --assume-open-loop --fault ps2_stuck_low
```

Mỗi lần chạy chọn thư mục mới, đổi 01 thành 02 nếu đã tồn tại. Lỗi đồ họa thì thử bỏ --view; lỗi đường dẫn thì kiểm tra tên file. HOLD không phải lỗi cài đặt. Không sửa ngưỡng để ép một bài thử báo thành công.

Bài tập trước khi chuyển sang chức năng tiếp theo

Mở summary.json, tìm trạng thái/lý do và số mẫu. Tìm PS1/PS2 đổi mức trong signals.jsonl; tự tính 0,235 chia chênh thời gian. Đối chiếu với estimated_transit_speed_m_s. Chỉ mở ground_truth.jsonl khi cần chấm sai số, không dùng nó làm đầu vào điều khiển.

Bản đồ học tiếp

Đọc theo thứ tự: signals.py → sensor_model.py → estimator.py → sequence.py → run_twin.py. Kiểm tra bằng test_signals.py và test_sequence.py; chạy check_twin.py để thử đủ 12 trường hợp. Hướng dẫn chi tiết: docs/dt_estimator_sequence.md.

Phần đã có: mô hình vật lý, tín hiệu, log/phát lại, ước lượng một phôi và máy trạng thái baseline. Phần tiếp theo: hiệu chuẩn và phản hồi còn thiếu; trễ/mất gói; chẩn đoán bất thường; mô phỏng dự báo độc lập và chọn chính sách tái cấu hình. **Chưa huấn luyện ML/RL; chưa nối mạng với PLC; chưa chứng minh tăng năng suất.**

Tự kiểm tra: Bạn đã nắm lõi khi trả lời được điều gì?
Đáp án: Máy biết gì từ cảm biến, đang giả định gì, có thể sai bao nhiêu, chuyển bước vì điều kiện nào và vì sao phải HOLD khi thiếu bằng chứng.

Đối chiếu: run_twin.py --help; check_twin.py; test_signals.py; test_sequence.py